

Stock Forecasting and Portfolio Optimization

A Complete Guide for Beginners and Interview Preparation

Elevator Pitch: This project builds an end-to-end quantitative trading and asset allocation pipeline. It downloads 19 years of daily stock prices for 61 S&P 500 equities, creates stationary technical features, reduces features via Principal Component Analysis (PCA), and trains a deep learning LSTM model to predict prices 22 days out. Finally, these predictions are optimized using a sparse L1-regularized mean-variance allocator, achieving superior risk-adjusted returns (Sharpe ratio) over Moving Average and Linear Regression baselines.

1. Core Financial Concepts (The Foundation)

Before diving into machine learning, it is critical to understand the financial data layer. Stock prices themselves are notoriously hard to predict directly because they are non-stationary (their mean and variance change over time). To train stable models, we must engineer features that represent relative changes.

Key Features Computed:

- **Daily Returns:** The percentage change in Close price from day $t-1$ to t . It represents the daily growth rate of the asset.
- **20-Day Returns:** The percentage change in Close price over the last month (20 trading days). This captures medium-term stock momentum.
- **20-Day Volatility:** The rolling standard deviation of daily returns over the last 20 days. This measures the asset's risk/uncertainty.
- **Rolling Z-Scores:** Normalized values computed by subtracting the 252-day (1 trading year) rolling mean and dividing by the rolling standard deviation.

Interview Concept - Non-Stationarity: Financial time series models must be stationary to ensure that the statistical properties (mean, variance) are constant over time. Z-scoring daily returns and volatilities over a rolling window removes trends, ensuring the LSTM learns structural relationships rather than arbitrary nominal changes.

2. Dimensionality Reduction & PCA

With 61 stocks and 11 features per stock, our model receives 671 inputs every day. Feeding this many highly correlated features into a neural network causes overfitting and slow training. We resolve this using Principal Component Analysis (PCA).

Why PCA is crucial in Finance:

- **Multicollinearity Removal:** Equity markets are highly co-dependent (e.g., tech stocks move together). PCA decorrelates the features by projecting them onto orthogonal principal axes.
- **Noise Reduction:** Most of the 671 features contain redundant information or noise. By selecting the top 61 principal components, we compress the data while retaining ~95% of the information variance.
- **Data Leakage Prevention:** PCA scalers are fit ONLY on the training dataset. We apply `transform` on the test set using the train-fitted PCA, ensuring future test information never leaks into the training phase.

3. Deep Learning Price Forecasting (PCA-LSTM)

Long Short-Term Memory (LSTM) networks are a specialized type of Recurrent Neural Network (RNN) designed for learning sequence relationships without suffering from vanishing gradients.

Model Architecture:

- **Sequence Prep:** Input sequences are shaped as `(batch_size, 252, 59)`, representing 252 days of historical daily PCA features across all stocks.
- **LSTM Layer 1:** 800 units, returns sequences. This extracts low-level chronological patterns.
- **LSTM Layer 2:** 600 units, collapses sequence. This aggregates temporal information into high-level features.
- **Dense Layer 3 & 4:** 2 Dense layers of size `1298` (22 days * 59 stocks) with ReLU and Sigmoid activations. The model outputs a flattened array representing the forecasted prices for all 59 stocks over the next 22 days (1 trading month).

4. Sparse L1-Regularized Portfolio Optimization

After obtaining predicted returns for the next month, we construct our portfolio using a regularized mean-variance optimizer to find the optimal stock weights (w).

The Objective Function:

We solve the following optimization problem at the beginning of each month:

Minimize: $- [w^T \mu - \lambda_1 (w^T \Sigma w) + \lambda_2 ||w||_1]$

Where:

- w : Vector of stock weights.
- μ : Expected predicted stock returns.
- Σ : Expected covariance matrix.
- λ_1 : Risk-aversion coefficient (0.5).
- λ_2 : L1-regularization coefficient (2.0) enforcing sparsity.

Constraints & Real-world Bounds:

- **Long-only constraint:** Bounded weights between 0 and 1 ($0 \leq w_i \leq 1$), preventing short-selling.
- **Fully invested constraint:** Sum of weights must equal 1 ($\sum w_i = 1$), ensuring all capital is allocated.
- **L1-Regularization (Sparsity):** Instead of allocating tiny fractions to all stocks, the L1 penalty forces weights of less-favorable stocks to exactly zero. This selects a concentrated basket of the top 5-10 stocks, lowering transaction fees and complexity.

5. Rolling Backtests and Bug Fixes

To test how our strategies perform in real-world conditions, we simulate a monthly rebalanced portfolio over a 5-year rolling window.

The Sharpe Ratio Bug Fix:

In the original repository's codebase, the Sharpe ratio was calculated as:

```
sharpe_ratio = portfolio_actual_returns / np.std(portfolio_actual_variance)
```

Since `portfolio\_actual\_variance` is a scalar float value, its standard deviation is always 0. This led to a division by zero warning, outputting `inf` or `-inf` Sharpe ratios in their backtest logs.

Our Correction: We implemented the true Sharpe ratio, dividing the mean daily return by the daily standard deviation (square root of variance):

```
Corrected Sharpe = portfolio_actual_returns / sqrt(portfolio_actual_variance)
```

6. Key Interview Questions & Answers

Q1: Why did you use PCA before the LSTM?

Answer: Financial features are highly collinear and noisy. Feeding 671 raw features directly into an LSTM causes overfitting and slows down training. PCA decorrelates inputs and reduces the dimensions to 61 components, removing multicollinearity while retaining 95% of the information variance.

Q2: What is the benefit of adding an L1 penalty to portfolio optimization?

Answer: A standard Markowitz optimizer creates highly fragmented portfolios with tiny weights across all assets. Adding an L1 penalty ($|w|_1$) drives the weights of weaker assets to exactly zero, resulting in a sparse portfolio. This reduces transaction costs and concentrates capital in the strongest ideas.

Q3: What was the main technical issue you identified in the original repository?

Answer: The original code calculated the Sharpe ratio by dividing returns by the standard deviation of a scalar variance, which resulted in a divide-by-zero warning and infinite values. We corrected this by using the square root of the portfolio variance to calculate the true Sharpe ratio.